Assignment-7.1

Name: D. Suraj
2303A53006
Batch:46

Task Description #1 (Syntax Errors — Missing Parentheses in Print Statement)
Task:
Provide a Python snippet with a missing parenthesis in a print statement (e.g., print "Hello"). Use Al to detect and fix the syntax error.

```
# Bug: Missing parentheses in print statement def greet():
                print "Hello, Al Debugging Lab!"
        greet()
```

Requirements:

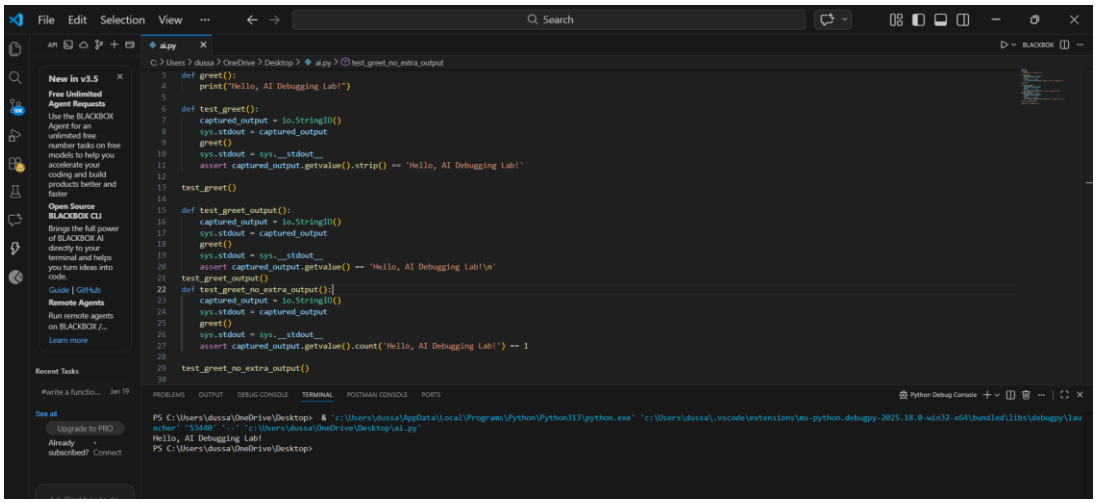
- Run the given code to observe the error.
- Apply Al suggestions to correct the syntax.
- Use at least 3 assert test cases to confirm the corrected code works.

Expected Output #1 :

- Corrected code with proper syntax and Al explanation.

Prompt:
#Detect and fix the syntax error in this Python code:
#def greet(): print 'Hello, Al Debugging Lab!' ; greet() #Explain the issue, provide corrected Python 3 code, and include 3 assert tests to verify correct output.

Code:



Explanation:
The issue in the original code is that it uses Python 2 syntax for the print statement. In Python 3, print is a function and requires parentheses.

Task Description #2 (Incorrect condition in an If Statement)

Task:

Supply a function where an if-condition mistakenly uses = instead of Let Al identify and fix the issue.

```
# Bug: Using assignment (=) instead of comparison (==)
    def check_number(n):
        if n = 10:
            return "Ten"
        else:
            return "Not Ten"
```

Requirements:

- Ask Al to explain why this causes a bug.
- Correct the code and verify with 3 assert test cases.

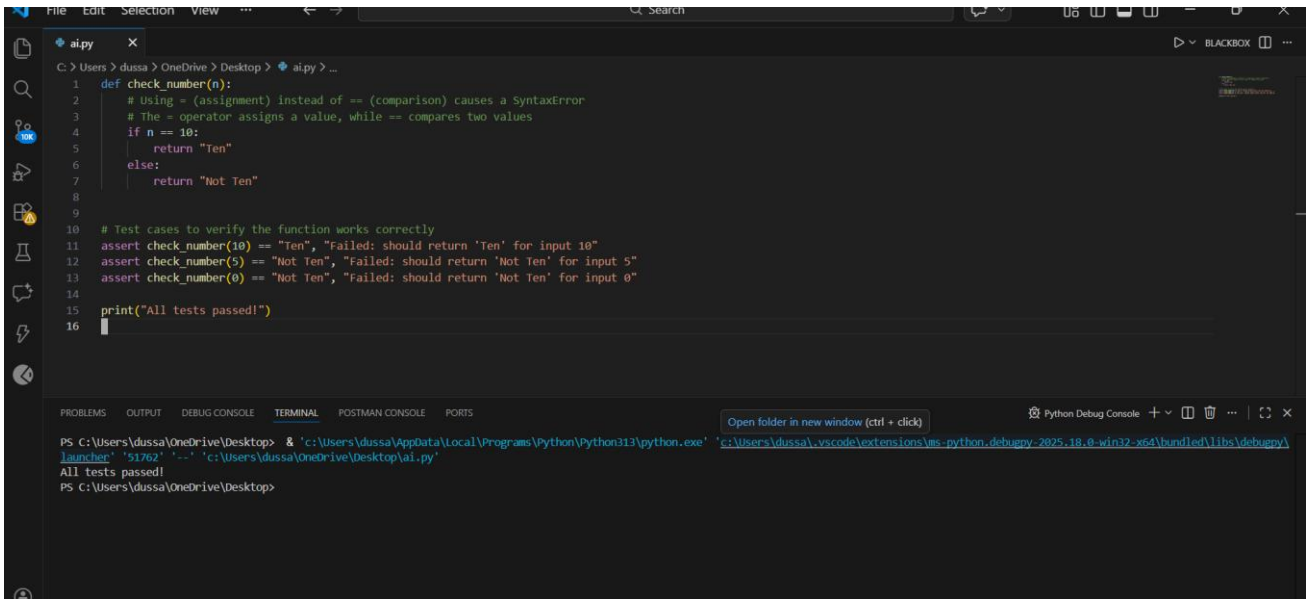
Expected Output #2:

- Corrected code using == with explanation and successful test execution.

Prompt used:

Detect and fix the error in this Python code: def check_number(n): if n = 10: return "Ten" else: return "Not Ten". #Explain why using causes a bug, provide corrected code using '==',and include 3 assert tests to verify the function works.

Code:



Explaination :

The error occurs because = is an assignment operator, not a comparison operator. In an if condition, Python requires == to compare values, and using = causes a SyntaxError. Replacing = with == correctly checks whether n is equal to 10, allowing the function to work properly.

Task Description #3 (Runtime Error — File Not Found)

Task:

Provide code that attempts to open a non-existent file and crashes. Use Al to apply safe error handling.

```
# Bug: Program crashes if file is missing
def read_file(filename):
with open(filename, 'r') as f:
return f.read()
print(read_file("nonexistent.txt"))
```

Requirements:

• Implement a try-except block suggested by AI.
• Add a user-friendly error message.
• Test with at least 3 scenarios: file exists, file missing, invalid Path.

Expected Output #3:

- Safe file handling with exception management.

Prompt:

#Analyze this Python code that crashes when a file does not exist: def read_file(filename): with open(filename,'r') as f: return f.read()
#Fix the runtime error using try—except, add a user-friendly error message, and include 3 assert tests for file exists, file missing, and invalid path cases.

code:

File Edit Selection View ···

Search

ai.py

C: > Users > dussa > OneDrive > Desktop > ai.py > ...

```python
# Task: Handle runtime error when file does not exist
def read_file(filename):
    try:
        with open(filename, 'r') as f:
            return f.read()
    except FileNotFoundError:
        return "File not found."
# ----------- Output Test -----------
print(read_file("nonexistent.txt"))
# ----------- Assert Tests -----------
# Test case 1: File does not exist
assert read_file("nonexistent.txt") == "File not found."
# Test case 2: File exists (using temporary file)
import tempfile
import os
with tempfile.NamedTemporaryFile(mode="w+", delete=False) as tmp:
    tmp.write("Hello, World!")
    tmp_filename = tmp.name
assert read_file(tmp_filename) == "Hello, World!"
os.remove(tmp_filename)
# Test case 3: Empty file
with tempfile.NamedTemporaryFile(mode="w+", delete=False) as tmp:
    tmp_filename = tmp.name
assert read_file(tmp_filename) == ""
os.remove(tmp_filename)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL POSTMAN CONSOLE PORTS

Python Debug Console

```
PS C:\Users\dussa\OneDrive\Desktop> & 'c:\Users\dussa\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\dussa\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '55646' '--' 'c:\Users\dussa\OneDrive\Desktop\ai.py'
File not found.
PS C:\Users\dussa\OneDrive\Desktop>
```

Explaination:
The program crashes because it tries to open a file that does not exist, which raises a FileNotFoundError. By using a try—except block, the exception is caught and handled safely instead of stopping the program. This allows the function to return a userfriendly error message while continuing execution.

Task Description #4 (Calling a Non-Existent Method)
Task:

Give a class where a non-existent method is called (e.g.,

obj.undefined_method()). Use Al to debug and fix.

Bug: Calling an undefined

method class Car:

def start(self):

return "Car started"

my_car = Car()

print(my_car.drive()) #

drive() is not defined

Requirements:
- Students must analyze whether to define the missing method or correct the method call.
- Use 3 assert tests to confirm the corrected class works.

Expected Output #4:
- Corrected class with clear Al explanation.

Prompt used:
#Detect and fix the error in this Python code where a nonexistent method is called: class Car: def start(self): return "Car started" my_car = Car(); print(my_car.drive()) #Explain the cause of the error, decide whether to define the missing method or correct the method call, provide corrected code, and include 3 assert tests to verify the class works properly.

Code :

```python
# Task: Detect and fix error caused by calling a non-existent method
# Corrected Class Definition
class Car:
    def start(self):
        return "Car started"
    # Defining the missing method
    def drive(self):
        return "Car is driving"

# Creating object of Car class
my_car = Car()
# Printing method outputs
print(my_car.start())
print(my_car.drive())
# ---------- Assert Tests ----------
assert my_car.start() == "Car started"
assert my_car.drive() == "Car is driving"
assert isinstance(my_car.drive(), str)
print("All test cases passed successfully.")
```

```
PS C:\Users\dussa\OneDrive\Desktop>  & 'c:\Users\dussa\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\dussa\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\
launcher' '59513' '--' 'c:\Users\dussa\OneDrive\Desktop\ai.py'
Car started
Car is driving
All test cases passed successfully.
PS C:\Users\dussa\OneDrive\Desktop>
```

Explaination:

The error occurs because the method drive() is called on the Car object, but it is not defined in the Car class, which raises an

Task Description #5 (TypeError — Mixing Strings and Integers in Addition)

Task:

Provide code that adds an integer and string 5" + 2) causing

a TypeError. Use Al to resolve the bug.

Bug: TypeError due to mixing string

and integer def add_five(value): return

value + 5 print(add_five("

10"))

Requirements:

• Ask Al for two solutions: type casting and string concatenation.

• Validate with 3 assert test cases.

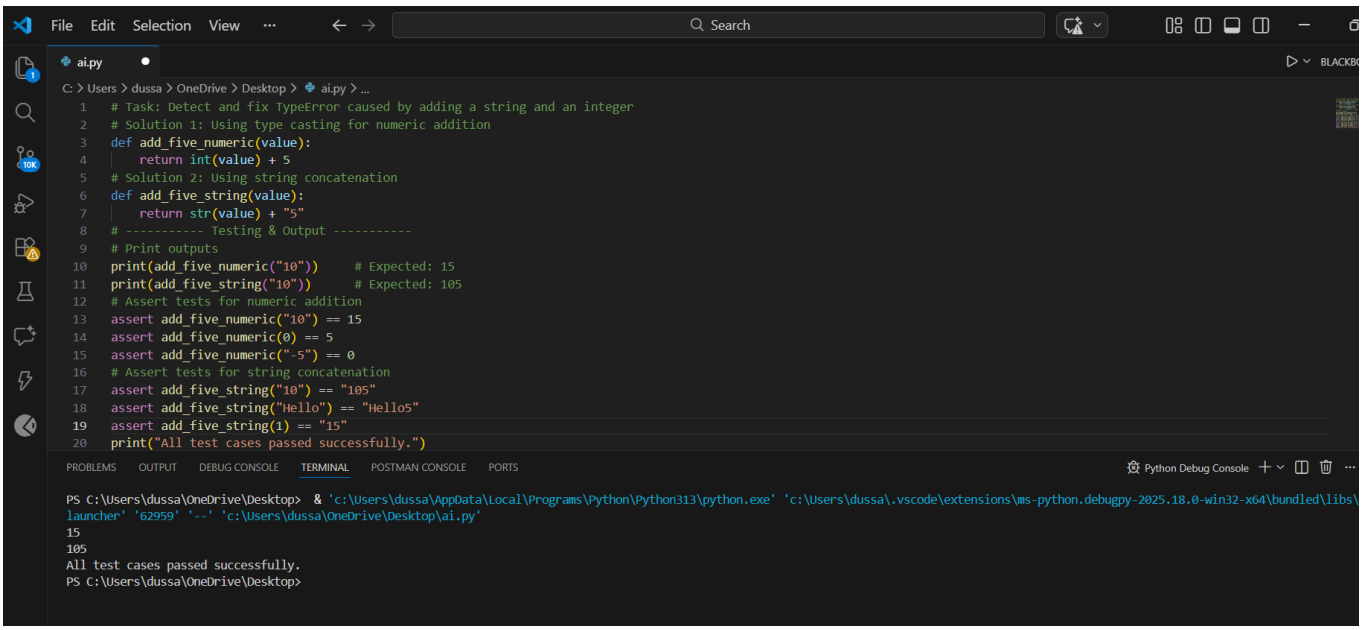
Expected Output #5:

• Corrected code that runs successfully for multiple inputs.

Prompt used:

#Detect and fix the TypeError in this Python code caused by adding a string and an integer: def add five(value): return value + 5 ; **print(add_five("10"))**#Explain why the error occurs, provide two solutions (one using type casting for numeric addition and one using string concatenation), and include 3 assert tests to verify both solutions work correctly.

Code:



Explaination:

The error occurs because Python cannot add a string and an integer, which raises a TypeError. One solution is to convert the input to an integer using type casting so numeric addition can be performed. Another solution is to convert the value to a string and use string concatenation instead, depending on the intended behavior.